МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ,
СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М. А. БОНЧ-БРУЕВИЧА»
(СПбГУТ)
Санкт-Петербургский колледж телекоммуникаций им. Э. Т. Кренкеля

ОТЧЕТ

по практической работе
учебной дисциплины «Технология разработки программного обеспечения»

на тему

«Приложение «Планировщик задач»»

Выполнил студент группы 541
Левушкин Никита Денисович

Руководитель: Кривоносова Н.В.

Санкт-Петербург

1. Реализованные требования

Разработано приложение для личных и совместных задач. Сервер написан на C# / ASP.NET Core, данные сохраняются в SQLite через EF Core. Клиент работает в браузере и не требует отдельной установки.

Требование	Реализация
Создание задачи	Название, необязательное описание, дата и время срока. Пустое название отклоняется сервером.
Просмотр	Поиск по названию, активные/выполненные/все задачи, сортировка по сроку или времени создания.
Изменение состояния	Редактирование, завершение, возврат в активные и удаление. Повторное завершение не создаёт дубль.
Доступ	Владелец управляет задачей и её доступом. Приглашённому зарегистрированному пользователю выдаётся чтение либо редактирование.
Авторизация	Access JWT проверяется на сервере. Refresh token хранится в HttpOnly-cookie, в базе остаётся его хеш. При обновлении token заменяется.
Напоминания	Worker проверяет сроки каждые 5 секунд. Сообщение сохраняется в списке напоминаний за 30 минут до срока либо после запуска при пропущенном сроке.
Сохранение	Перезапуск сервера не удаляет пользователей, задачи, доступы и напоминания.

Структура

Models.cs содержит модель и PlannerDb, Auth.cs работает с токенами, Reminders.cs содержит INotifier и фоновую проверку. API в Program.cs проверяет права для каждой операции. HTML, CSS и JavaScript размещены в wwwroot.

Уникальный ключ напоминания включает задачу, владельца и срок. Поэтому повторная проверка не добавляет одинаковое сообщение. Если срок перенесён, допускается новое напоминание.

2. Интерфейс и запуск

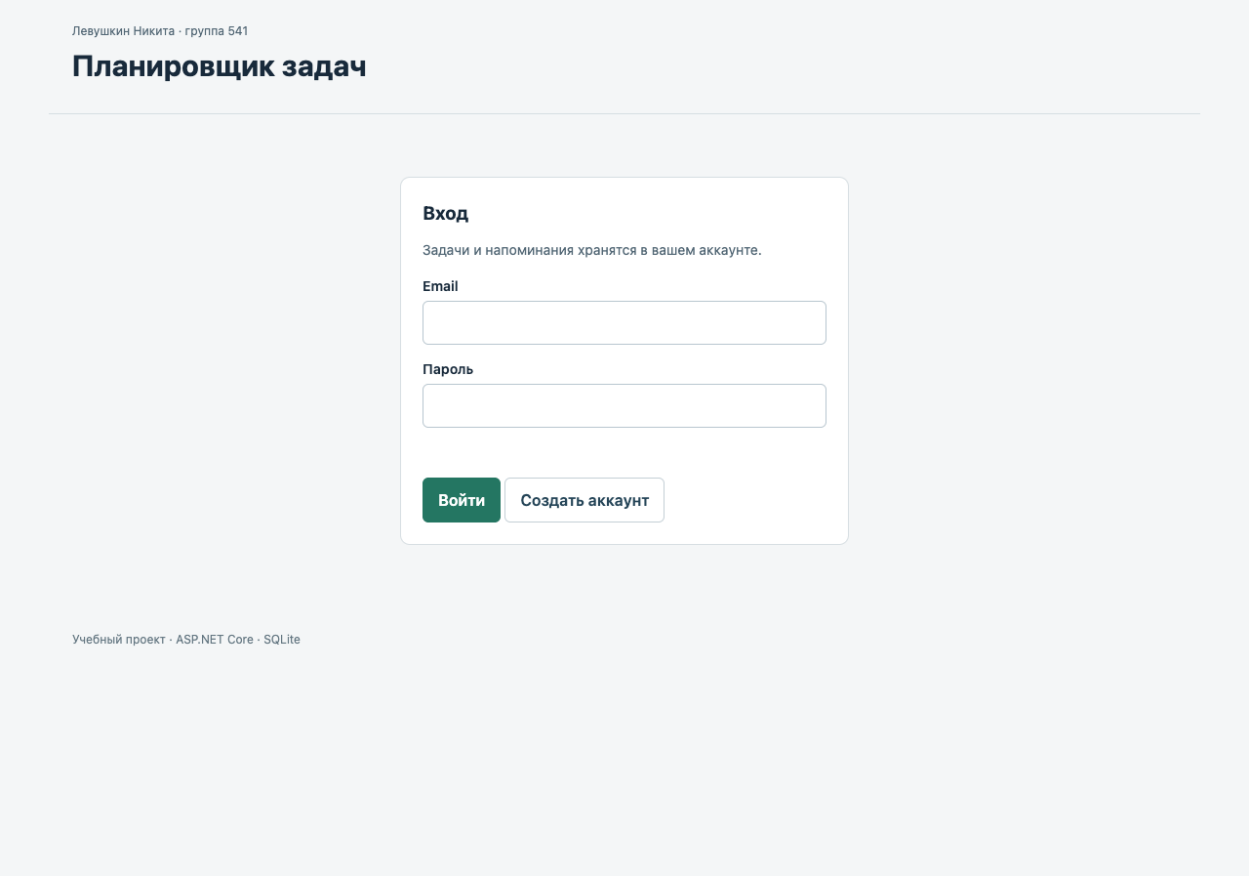


Рисунок 1. Настоящая страница входа запущенного приложения.

Запуск из исходников

Установить .NET 10 SDK, открыть корень TaskPlanner и выполнить:

```
dotnet run --project src/TaskPlanner --urls http://localhost:5080
```

Открыть http://localhost:5080 и создать аккаунт. Email используется как логин, без отправки писем. В готовой Windows-сборке нужно распаковать архив и запустить «Запустить.cmd»; .NET включён в комплект.

Основные действия доступны из списка задач. Ошибки формы показываются рядом с полями, введённый текст сохраняется. В отдельном блоке находятся напоминания и отметка «Прочитано».

[GitHub] Исходники, инструкция запуска и автоматические проверки
https://github.com/Nikita-Levuskin/Task-Planner-541

3. Проверка приложения

Проверка	Подтверждённый результат
Сборка Release	Успешно, без ошибок и предупреждений.
Тесты ядра: 12	Проверены JWT, истечение срока, неверная подпись, случайность refresh token, напоминания, перенос срока, повторы и каскадное удаление.
HTTP-тесты: 16	Проверены CRUD, фильтры, сортировка, изоляция пользователей, общий доступ, refresh rotation, защита от чужого Origin, выход и восстановление после перезапуска.
Браузерная проверка	Регистрация, вход, создание, завершение, изменение, удаление и восстановление сессии после обновления страницы выполнены через интерфейс.
HTML-инъекция	Название с HTML отображается как текст. Внедрённый элемент не создаётся, JavaScript не выполняется.
Поставка	Подготовлены исходники, Windows x64-сборка, README, тесты и workflow GitHub Actions.

Границы реализации

Напоминания сохраняются в приложении. Всплывающее уведомление браузера требует разрешения пользователя и открытой страницы. SMTP-рассылка не используется. После выхода refresh token отзывается сразу, а ранее выданный access token действует до истечения своего короткого срока.

Для учебного запуска сервер доступен только локально. Публичное размещение требует HTTPS, отдельного ключа подписи, резервного копирования и настройки эксплуатации.

Вывод

Реализован полный цикл работы с задачей: создание, изменение, завершение, совместный доступ и удаление. Проверки выполняются как на уровне сервисов, так и реальными запросами к серверу.

Источники

[1] Microsoft Learn. ASP.NET Core
https://learn.microsoft.com/aspnet/core/

[2] Microsoft Learn. EF Core SQLite
https://learn.microsoft.com/ef/core/providers/sqlite/

[3] Microsoft Learn. JWT bearer authentication
https://learn.microsoft.com/aspnet/core/security/authentication/configure-jwt-bearer-authentication